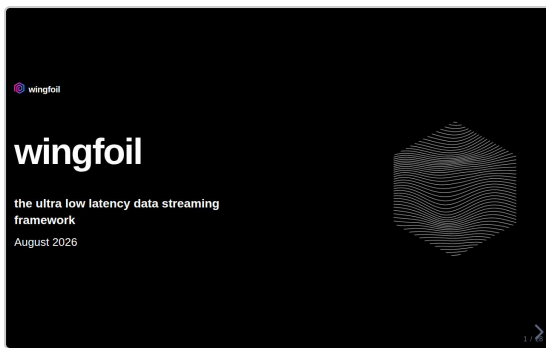


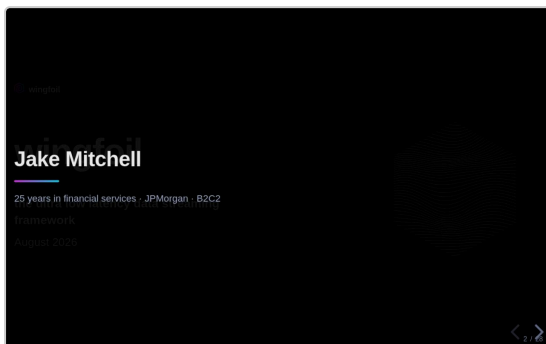
wingfoil — speaker notes

28 slides · printed fallback for the reveal.js speaker view (press S)



1 / 28

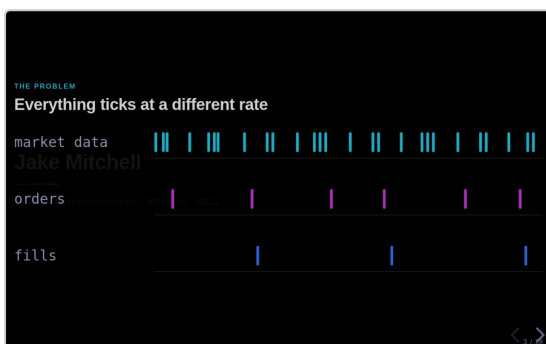
- Thanks for coming. Jake Mitchell. Open source project — **wingfoil**.
- Three parts:
- **1** — background: myself, the problem, and why you need wingfoil to solve it.
- **2** — **Nitro**. The feature we released last week. Why we wanted it. Why it meant rewriting the entire framework. The pattern we found, and what it delivered.
- **3** — what's next.



2 / 28

Jake Mitchell

- 25 years in financial services.
- Mostly as a quant at JPMorgan — building electronic trading systems for options: across interest rates, FX, precious metals.
- Now at B2C2, working for their crypto options desk.
- I learnt what the most effective patterns were for building these kinds of systems.
- I wanted to bring them into the open source world. And I wanted to use **Rust** to build it.



3 / 28

THE PROBLEM

Everything ticks at a different rate

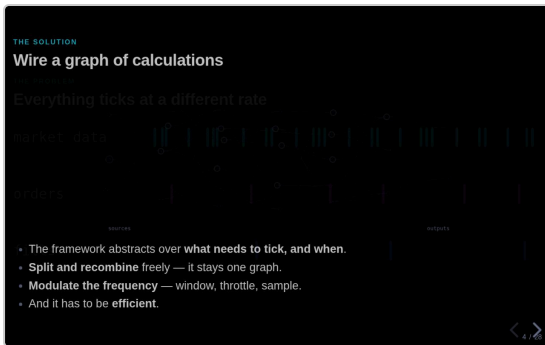
- The problem.
- You're building something that reacts to information that is arriving at **different rates**.
- For example — market data ticking very frequently; orders and fills at a different frequency.
- This could be across tens of thousands of assets.
- And you need to react fast — at micro or even **nanosecond** time scales.

4 / 28

THE SOLUTION

Wire a graph of calculations

- Wire a graph of calculations.
- Have a framework that **abstracts over what needs to tick at what time**.
- That allows you to split and recombine.
- And to modulate their frequency — for example window or throttle.
- And is **efficient**.



5 / 28

THE SURFACE

Wire it, build it, run it

- This is what it looks like in practice.
- This example is a simple linear pipeline, with every node ticking on each engine cycle.

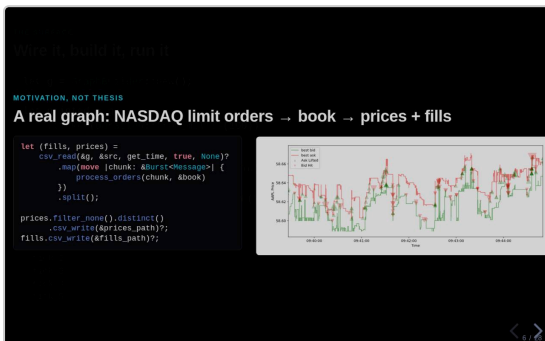


6 / 28

MOTIVATION, NOT THESIS

A real graph: NASDAQ limit orders → book → prices + fills

- Here is a more interesting example: a **matching engine**.
- Replay limit orders from file.
- Maintain an order book over time.
- Produce top-of-book prices and fills.
- **Three different frequencies of data** — many messages share a timestamp, top of book changes less often, trades are sparser still. Nothing is coalesced or dropped to make them agree.



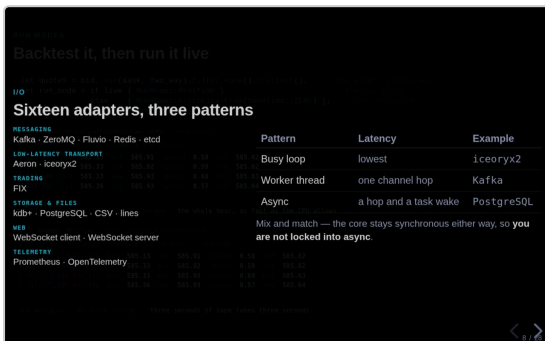
Backtest it, then run it live

- Same wiring. Same graph. One line changes — the run mode.
- Top: the replay. It reads **no clock at all** — engine time is source-driven, so it goes as fast as the CPU walks the graph. An hour of AAPL in a fraction of a second.
- Bottom: same program, live. Rows arrive when the market gives them to you, and you can see it waiting in between.
- The quotes are **identical** across the two. Only the clock column differs — replay counts from zero, live is epoch wall-clock.
- That's the determinism: a backtest you can re-run and get the same answer.
- Second clock, separately: a wall-clock snap for latency stamping. Never branch business logic on it.
- And a live source **refuses a historical run at wiring time**, not at 3am — historical collects up front, so a live tail would deadlock.
- *[Live panel is slowed 12x, and says so — the five real gaps span a quarter of a second.]*



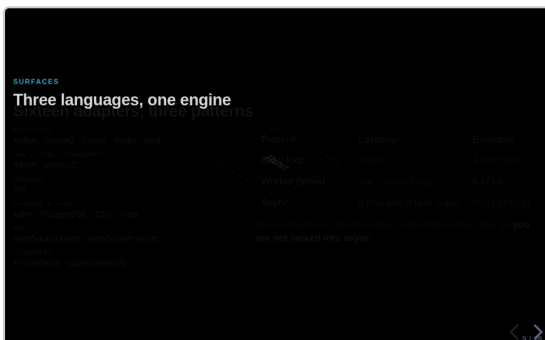
Sixteen adapters, three patterns

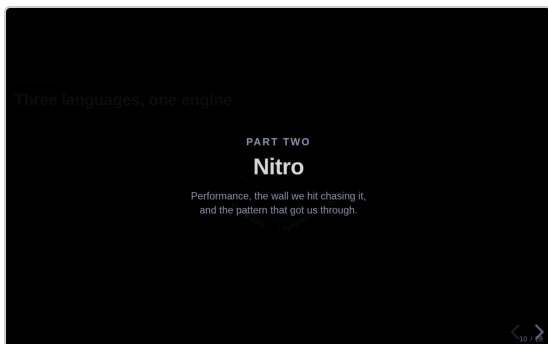
- We've built a bunch of I/O adapters — FIX, kdb+, iceoryx2, and more.
- They use these three I/O patterns:
- **Busy spin** — for latency-critical, realtime use cases.
- **Dedicated worker thread.**
- **Async** — but only where you need it, at the graph edge. The graph itself is synchronous.
- You can **mix and match** — choose the pattern that works for your use case.



Three languages, one engine

- **Rust** — the best language to work with.
- Also **Python** bindings, for those that have yet to make the jump.
- **TypeScript** integrations, to stream into the browser.



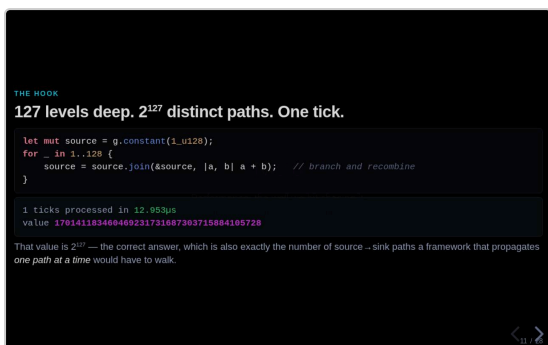


10 / 28

PART TWO

Nitro

- That's part one.
- Part two — Nitro.



11 / 28

THE HOOK

127 levels deep. 2^{127} distinct paths. One tick.

- 128 levels deep. Branches and recombines at every level.
- So distinct source → sink paths **double every level**.
- At the bottom: 2^{127} paths.
- That number on screen is the answer — **and** the path count.
- **Thirteen microseconds. One tick.**
- *[Slow down. Let it sit.]*



12 / 28

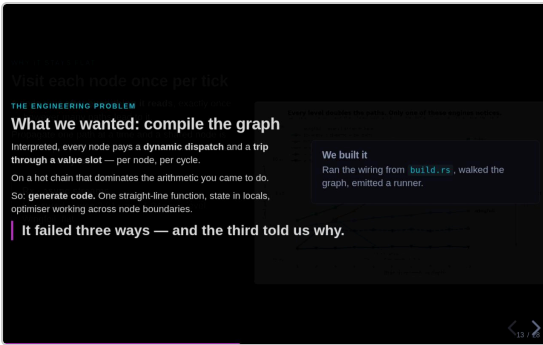
WHY IT STAYS FLAT

Visit each node once per tick

- The reason is boring, and that's the point.
- Every node runs once per tick, after everything it reads. However many paths lead in.
- Push one path at a time — reactive libraries, async streams — and you re-visit shared nodes per path. Cost doubles every level.
- Measured slopes: **2.01** and **1.94**. Ours: flat.
- *[Get to the caveats first: rxrust figure is HALVED from their raw number (their bench emits twice per sample). Tokio target is recursive Box::pin over a ready leaf — honest for per-path propagation, not the best async could do. **The claim is the slope, not the multiplier.**]*

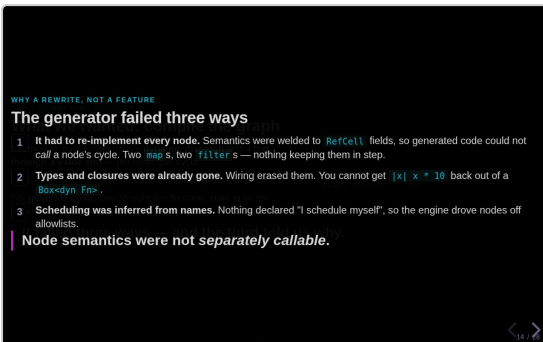
What we wanted: compile the graph

- Interpreted engine is fine, but there's overhead I don't love.
- Per node, per cycle: a **dynamic dispatch** and a **trip through a value slot**.
- On a hot chain that's bigger than the arithmetic you came to do.
- Obvious answer: **generate code**. One straight-line function, state in locals, optimiser working across node boundaries.
- Good plan. Every ingredient exists.
- So we built it. Wiring at build time, walk the graph, emit a runner.



The generator failed three ways

- **It failed. Three ways.**
- **1** — had to re-implement every node. Semantics welded to `RefCell` fields, so generated code couldn't *call* a node's cycle. Emitter carried its own copy, as strings. Two maps, two filters. Nothing keeping them in step.
- **2** — types and closures already gone. Types came back as name strings we parsed. You cannot get a closure back out of a `Box<dyn Fn>`. Workaround: user retypes every closure in a side table, with no compiler check the two agree.
- *[That one usually gets a laugh. Let it.]*
- **3** — nothing declared how it wanted scheduling. Engine drove nodes off **allowlists of names**.
- We deleted it. *[Pause.]*
- **The problem was never the generator. Node semantics weren't separately callable.** No cleverness in an emitter fixes a design with nothing to emit a call *to*.



Inside-out

- The pattern that got us out.
- **The tension:** encapsulation says bundle state with behaviour. Single responsibility says one reason to change. Follow the first far enough and you break the second.
- Our node was computation *and* storage *and* plumbing. Three reasons to change, one object.
- **The resolution:** keep logic and types encapsulated. Move **ownership** out.
- *[Point at the arrow.]* The engine passes in state it *can't read*, to logic it *can't see inside*.
- **Ownership moved out. Encapsulation didn't.**
- **The op owns what and how. The engine owns where and when.**
- *["Isn't that just an ECS?" — yes, and reducers, gen_server, React hooks. Same family. Ours is the combination: reducer-shaped semantics + opaque associated types + a const for scheduling. That's what gets you two engines.]*

THE ONE DECISION EVERYTHING FOLLOWS FROM

Semantics became an associated function

```

trait Op {
  type Cfg;           // construction-time config; closures live here
  type State;         // engine-owned mutable state
  type In<'a>;         // typed inputs, passed in per cycle
  type Out;           // the produced value
  const ACTIVATION: Activation;

  fn cycle(cfg: &mut Self::Cfg, state: &mut Self::State,
    input: Self::In<'a>, ctx: &mut Ctx<'a>) -> Result<Tick<Self::Out>>;
}

```

Read what is *absent*. Nothing here says how state is stored, where inputs come from, or who calls it.

Semantics became an associated function

- Four associated types and a function.
- Cfg (closures live here) · State (engine owns) · In (typed, per cycle) · Out. Plus a const for scheduling.
- **Notice what's missing.** Nothing says where state lives. Nothing says who calls it.
- It's a free function over explicit arguments.
- *[Return type, if asked: Tick has THREE states. Value ticks downstream, Quiet does nothing, Silent updates the value without ticking — what delay needs.]*

THE SAME NODE, BOTH WAYS

The node changed completely. The wiring barely moved.

legacy — the node owns everything

```
fn connective = (upstream, output = value: f64) {
  impl NodeBuilder for ScaleStream {
    fn cycle(&mut self, _state: &mut GraphState)
      -> Result<bool> {
      {
        self.value =
          self.upstream.peek_value() * self.factor;
        Ok(true)
      }
    }
  }
}
```

owns: its upstream handles, its state, its output slot

```
let count = ticker(Duration::from_millis(10))
count();
const _ = &node::new(RealTime, Number::cycles(100)37);
```

Read what is absent. Nothing here says
come from, or who calls it.

now — the engine owns everything

```
fn op(hold = scale, f64mut) {
  impl Op for Scale {
    type State = f64; // owned
    type State = f64; // engine-owned
    type OutAn = f64 * f64; // passed in
    type Out = f64;
    const ACTIVATION: Activation = Activation::NONE;

    fn cycle(&mut f64, _state: &mut (),
      _input: (&f64, _), _ctx: &mut State) ->
      -> Result<Tick<f64>> {
      {
        Ok(Tick::Value(input.0 * *ctx))
      }
    }
  }
}
```

owns: nothing — ctx, state and inputs all arrive as arguments

```
let g = GraphBuilder::new();
let count = g.ticker(Duration::from_millis(10))
count();
g.hold().run(&node::new(RealTime, Number::cycles(100)37);
```

- Same node, both ways.
- **Left:** &mut self, peeks a stored upstream handle, writes its own field. Owns its upstreams, its state, its output slot.
- **Right:** same node as an op. Owns **nothing** — everything arrives as an argument.
- Those two are unrecognisable.
- **Now the bottom row** — the wiring the user writes. Before and after.
- **One line different.** You hold a builder and make sources on it.
- We turned the engine inside out, and it cost anyone using it one line.

The node changed completely. The wiring barely moved.

WHAT IT BUYS

Two engines, one copy of the semantics

Engine	Who owns state	Dispatch
Interpreted	the builder's slots	one dyn call per node
Compiled	locals in a generated fn	monomorphized, inlinable

One implementation — which is why they cannot drift.

...and you choose the boundary, not the engine

A **nested island** mounts the compiled emission as *one* node of an interpreted graph. Not a third engine — which is why it lands between the two.

Two engines, one copy of the semantics

- Two engines out of one definition.
- Interpreted: state in the builder's slots, one dyn call per node.
- Compiled: state in locals of a generated fn, monomorphized, optimiser inlines through.
- **One implementation of what a node does.** They can't drift — nothing to drift from.
- Island: mount the compiled version as one node *inside* an interpreted graph. Hot core compiled, edges open.
- **Not a third engine — the seam between the two.** Which is why it lands between them on every benchmark.

THE FRONT DOOR

You write the wiring once, in plain Rust

```
wingfoil::nitro! {
  fn odds_events(g: &GraphBuilder) -> Stream<String> {
    let count = g.ticker(PERIOD).count();
    let is_even = count.map(|i| 1.is_multiple_of(2));
    let is_odd = is_even.map(|b| !b);
    let odd_str = count.filter(&is_odd).map(|i| format!("{i} is odd"));
    let even_str = count.filter(&is_even).map(|i| format!("{i} is even"));
    let labelled = odd_str.merge(&even_str);
    labelled
  }
}
```

Out comes `odds_events::interpreted()`, `odds_events::compiled()` and `odds_events::nested()` — from the same tokens.

The wiring function stays **valid**, **reviewable** Rust either way. That property was deliberately never traded away. `count` is read three times, so it is a shared apex node — the macro derives a real DAG, not a chain.

You write the wiring once, in plain Rust

- You write the wiring once. Plain Rust.
- Real function. Valid, reviewable, readable in a PR.
- Out the other side: interpreted, compiled, nested. **Same tokens.**
- `count` used three times → a shared node. This is a real DAG, not a chain.
- *[Constraint, if asked: wiring must be straight-line. Values and closure logic can be as procedural as you like — the TOPOLOGY can't depend on a runtime value, because compiled monomorphizes one local per node.]*

```
...AND THIS IS WHAT IT BECOMES
One function. One local per node. No dyn.

let mut __k = Kernel::new(run_node, run_for);

let mut __cfg_wf_anon_1 = PERIOD;
let mut state_wf_anon_1 = wf_op_ticker_seed_state(& __cfg_wf_anon_1);
let mut __v_wf_anon_1 = wf_op_ticker_seed_value(& __cfg_wf_anon_1);
// ... one cfg / state / value local per node, 11 nodes ...

let mut __dirty = [false; 11usize];
while __k.begin_cycle(&mut __dirty) {
  let __t_wf_anon_1 = /> activation /> && {
    let mut __ctx = Ctx::new(&mut __k, &mut __dirty);
    match __wf_op_ticker_cycle(&mut __cfg_wf_anon_1, &mut __state_wf_anon_1, &mut __ctx) {
      .context("node 0 (wf_anon_1: ticker) cycle")? {
        Tick::value(__val) => { __v_wf_anon_1 = __val; true }
        Tick::silent(__val) => { __v_wf_anon_1 = __val; false }
        Tick::quiet => false,
      }
    }
  }
  // ... 30 more, emitted in topological order ...
}

```

...AND THIS IS WHAT IT BECOMES

One function. One local per node. No dyn.

- What it turns into.
- Kernel on the stack. One local per node — cfg, state, value. Then a loop.
- *[Point at the call.] That's the same cycle function the interpreter calls.* Not a copy. A call.
- Wall number one, dead.
- Error context = the binding name *you* wrote, baked in as a static string. Costs nothing until something breaks.
- Whole expansion is committed. 1,730 lines. Don't take my word for it.

```
One function. One local per node. No dyn.

BOTH ENGINES ARE IN THE BINARY
Flip tiers without a rebuild

$ WINGFOIL_TIER=interpreted dual_node
0.000.000 odds/events "1 is odd"
0.010.000 odds/events "2 is even"
0.020.000 odds/events "2 is odd"
ran 10 cycles on the interpreted tier

$ WINGFOIL_TIER=compiled dual_node
0.000.000 odds/events "1 is odd"
0.010.000 odds/events "2 is even"
0.020.000 odds/events "2 is odd"
ran 10 cycles on the compiled tier

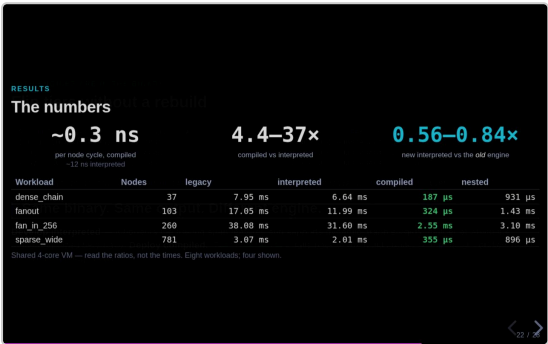
Same binary. Same output. Different engine.

Develop interpreted — it honours the tracing span features and supports dynamic graph surgery, neither of which the monomorphized tiers have. Deploy compiled. Tier::default() falls back to the build profile, so the usual workflow needs no call-site change at all.
```

BOTH ENGINES ARE IN THE BINARY

Flip tiers without a rebuild

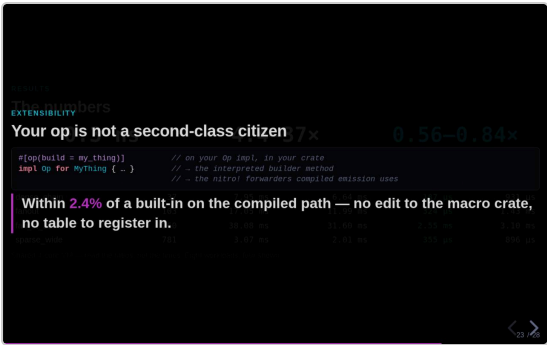
- Both engines are in the binary.
- Same binary, same output, different engine. **An environment variable.**
- Develop interpreted — proper tracing, surgery on a running graph. Neither of which compiled gives you.
- Deploy compiled.
- *[The two agreeing isn't a hope — a test runs the same graph through both and pins the outputs equal.]*



RESULTS

The numbers

- ~0.3 ns per node cycle compiled. ~12 ns interpreted.
- Compiled 4.4x to 37x, depending on graph shape.
- The one I care about most: **new interpreted beats the OLD engine. All eight workloads.**
- Because most graphs keep running interpreted — and a rewrite that trades everyday performance for a fast path nobody uses is a bad trade.
- *[Read the ratios, not the times. Shared cloud VM, measured back to back in one run.]*



23 / 28

EXTENSIBILITY

Your op is not a second-class citizen

- The one I doubted most.
- Attribute on *your* op, in *your* crate → interpreted builder method + compiled forwarders.
- **Within 2.4% of a built-in** on the compiled path.
- No edit to the macro crate. No table to register in.
- There *is* no per-op table. Yours and ours take the same road.

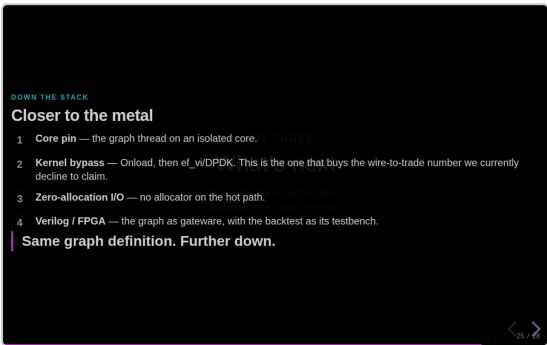


24 / 28

PART THREE

What's next

- That's Nitro.
- Last bit — where this goes.

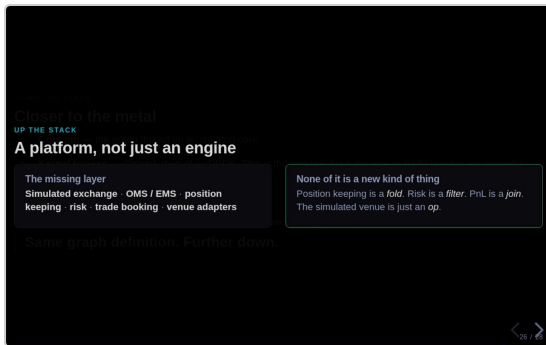


25 / 28

DOWN THE STACK

Closer to the metal

- **Core pin** — graph thread on an isolated core. Working implementation sits in an example; needs promoting into the runtime.
- **Kernel bypass** — Onload, then raw ef_vi/DPDK. Gets us a wire-to-trade number, which **I won't claim yet because I haven't measured it.**
- **Zero allocation** on the I/O path.
- **Verilog** — the graph as gateway, backtest as testbench. Exploratory. Don't oversell it.
- The point: **none of these change your wiring.** Same graph definition, further down. That's the payoff from part two.

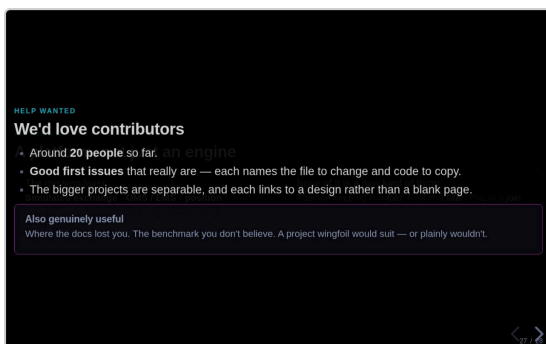


26 / 28

UP THE STACK

A platform, not just an engine

- Where most of the value is, for most people.
- **Be straight:** wingfoil is a compute engine. It is *not* a trading platform.
- Missing: simulated exchange, OMS, position keeping, risk, booking, more venue adapters.
- Simulator is the hard one — a naive fill-at-touch simulator lies to you, and you'll believe it.
- **The bet:** none of it is a new kind of thing. Position keeping is a *fold*. Risk is a *filter*. PnL is a *join*. The sim venue is just an *op* — RunMode picks.
- Ordinary graph vocabulary. Not a second framework bolted on top.

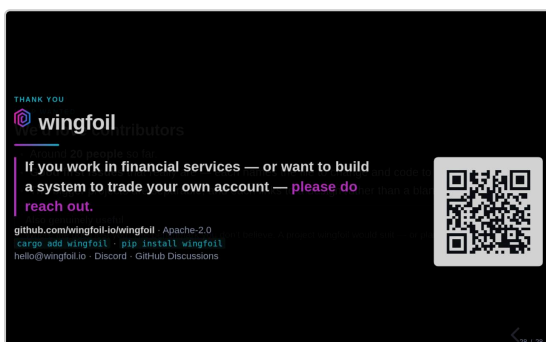


27 / 28

HELP WANTED

We'd love contributors

- The ask.
- ~20 contributors so far. I'd love more.
- **The good first issues are actually good first issues** — each names the file to change and code to copy. I know that label's been ruined elsewhere.
- Bigger projects are separable — one person each, and each links to a design not a blank page.
- Most useful isn't always code: where the docs lost you · which benchmark you don't believe · a project this would suit, or obviously wouldn't.



28 / 28

THANK YOU

wingfoil

- **If you work in financial services — or you're building something to trade your own account — please get in touch.** That's genuinely why I'm up here.
- Repo's on the QR. Thank you.
- *[Leave it up. Don't black the screen — let people scan.]*
- *[If it's quiet: the question I usually get is what happens when the graph shape isn't known until runtime. That's Project Lightning, and it's open.]*